Bot-ridden YT Gaming
YouTube Gaming is littered with bot-driven videos selling scams and cheats that try to steal your info. https://dgit.in/mar20-17

The Quantum Internet
With the advent of quantum computing, we're laying down the pipes for the quantum internet. https://dgit.in/mar20-18

payloads are injected into log files, which in turn gets executed when the logs are accessed through and already present vulnerability. For example, apache access logs are stored at '`/var/log/apache2/access.log`' and with a crafted HTTP request, this file can also be used to get remote code execution.

There is an ocean of web application vulnerabilities and explaining, even listing all of them, is beyond this article's scope. The ones we mentioned are the easiest to creep in your code and also the most damaging. Here are some resources which will expand your knowledge about web application security:

- https://dgit.in/owasptop10
- https://dgit.in/portswigger

Alright, now we have secured your interface. But the problems don't end here, you need to secure the backend too. Since today most of the websites are cloud-based and backend services are maintained by them unless there's some Oday (Oday vulnerabilities are unknown to the software vendors) vulnerability is uncovered, most of the backends are secure. We'll cover the misconfigurations that might allow hackers to wreak havoc, for the sake of the good ol' days of system admins screwing things up and people who still today like getting their hands dirty, but we'll keep it brief.

Major server misconfigurations:

- **Privilege Management:** This is the part where most people weaken their systems. Giving unnecessary permissions to services can lead to abuse of power once intruders break-in.
- **Unnecessary services:** By default, operating systems come with a lot of services, most of them are not required for a normal user. And if they are left to install, they can turn into weapons if encountered by hackers because they wouldn't have been properly configured because they weren't required in the first place.
- **Default credentials:** This exists solely for reason: humans are lazy. And believe us, there are a lot of servers running around the world with services having default credentials. Logging service, databases, routers, and whatnot. Just because one lazy afternoon the IT guy wasn't "feeling it".
- **Unpatched software:** Known vulnerabilities spread like wildfire in the hacker community. Older vulnerable software/binaries lingering in your system can lead to quick and efficient circumvention of your security measures since these days PoC (proof-of-concept, a working exploit that is developed during Oday discoveries) code is available due to public disclosure policies, which can prove lethal with little tweaking to make it compatible with the given system.

Now that you have the diseases listed and diagnosis ready, it's time to implement the cure.

THE BUG-SQUASHING:

Security solutions always span categories. There are certain procedures that ensure heightened security and minimise the overall damage to the system:

- **Sanitising inputs:** This is the holy grail, the "ramban", to make sure your systems aren't getting fooled by some script kiddie copy-pasting payloads. There's an old saying in security circles "Treat every input like it's malicious". Research major vulnerabilities, how they are exploited and create your own whitelist/blacklist counter-measures to allow/disallow the characters that are used in the payloads. Input sanitisation is the easiest and most effective solution against XSS, LFIs, and all other numerous injections.
- **Uninstallation of unnecessary services:** If something's not required, it should not be there, simple as that. Clutter leads to loose ends and you don't want that. (We sound like mobsters, don't we?)
- **Set up a honeypot:** Feeling experiment-ish? This is for you. A "honeypot" is a trap. A fake vulnerable subsystem of your actual web server used to lure the attacker and then hold him hostage. Honeypots contain dummy data without any damaging consequence, it's a "rabbit hole".
- **Isolation of environments:** Keep testing/development environment isolated from the production environment, so that the risk is localised. This ensures that even if someone is able to intrude in they won't be able escalate their powers and cause damage to the live website.
- **Use of automated scanners:** Even though using secure code practices is a good way to make your end result less prone to vulnerabilities, many little bugs creep past the human eye and persist in the code. That's where automated scanners come in. These are special tools, created specifically for scanning web applications and reporting exploitable content they find. Here's a list of popular ones:
  - Acunetix WVS (Commercial/Free)
  - NetSparker (Commercial)
  - Arachni (Open Source)
  - Vega (Open Source)
  - Qualys (Commercial)
  - Wapiti (Open Source)
- **Privilege Management:** If you remember devworx from December issue last year, we talked about how privilege mismanagement can lead to hackers compromising administrator accounts and complete system takeover. Which makes proper privilege and permissions configuration even more important. It is advisable to follow the "Principle of Least Privilege", according to which only the minimum required permissions are provided and nothing more. Remember to take care of rogue permission, which may seem benign, but may lead to privilege escalation attacks.
- **Log management:** Logging helps in monitoring the web application. Are any attempts being made to break in? Are the application spitting strange errors? All these answers are logged. Alongside logging, regular auditing of these logs is also crucial.
- **Software patches:** The simplest and easiest way to increase the security of your web system. Vendors are quick and efficient in releasing security updates for their software, all you have to do is regularly install them. Automation can be used to make this process something you can forget